INTERNSHIP DEFENSE & CODE EXECUTION GUIDE

CarCare System Manual

Line-by-Line Function, Button, Screen & Database Defense Reference

Project:	CarCare Auto Workshop Management
Developer:	Muhammad (Intern at ColonNext)
Architecture:	3-Tier (Presentation • Business • DataAccess)
Technologies:	Java Swing, FlatLaf 3.7.2, MySQL (XAMPP), NetBeans
Database:	carcare_db (9 Fully-Connected Active Tables)
Status:	Cleaned, Audited & 100% Compilation Verified

Table of Contents

1. Application Architecture & Master Navigation Flow — Page 2
2. Database Schema, Relationships & Foreign Keys — Page 3
3. How the Tabbed Navigation Works (Step-by-Step) — Page 4
4. Data Access Layer (DAO) — Every Function & SQL Query — Page 5
5. Business Logic Layer (BLL) & Domain Entities — Page 8
6. Presentation Layer — Screen-by-Screen Code Walkthrough — Page 10
7. Every Single Button Click Traced to Database (15 Flows) — Page 14
8. In-Depth Billing & Invoicing Mechanics — Page 17
9. Evaluator Defense Q&A (Questions & Exact Answers) — Page 19

1. Application Architecture & Master Navigation Flow

CarCare is built using the industry-standard **3-Tier Architecture**. This cleanly separates user interface, business logic, and database operations:

Layer	Package Location	Core Responsibility
1. Presentation Layer	`PresentationLayer.*`	JFrame and JDialog GUI screens designed in NetBeans Matisse, FlatLaf styling, user input capture, table rendering, button event handling.
2. Business Layer	`BusinessLayer.*`	Domain Entities (`Customer` , `Vehicle` , `Service` , `Bill`) and Business Logic Services (`Login_BLL` , `Vehicle_BLL` , `Customer_BLL`) containing input validations and business rules.
3. Data Access Layer	`DataAccessLayer.*`	Raw database interaction via JDBC (`PreparedStatement` , `ResultSet`), executing SQL queries against the MySQL `carcare_db` database.

System Launch Sequence

Here is the exact step-by-step process of how the application boots up when you run it:

1. **Entry Point:** Execution starts in `LoginForm.main()` (Line 250).
2. **Look & Feel Setup:** `FlatLightLaf.setup()` applies modern rounded cards, clean typography, and native OS anti-aliasing.
3. **Login Window Displays:** `LoginForm` appears on screen, centered vertically and horizontally.
4. **Authentication Flow:**
    - User types email and password, then clicks the **"LOGIN"** button (or presses Enter).
    - `LoginForm.handleLogin()` (Lines 39-63) extracts inputs and calls `Login_BLL.login(email, password)` .
    - `Login_BLL` trims strings, verifies non-empty, and delegates to `Login_DAO.validateLogin(email, password)` .
    - `Login_DAO` executes: `SELECT * FROM users WHERE email = ? AND password = ?` .
    - If true: `this.dispose()` destroys the login window and `new VehicleManagement_Screen().setVisible(true)` opens the master application window!
5. **Master Window Initialization:** `VehicleManagement_Screen` constructor runs:
    - `setupWindow()` : Sets title, dimensions (1100×700), navy blue background.
    - `setupTabs()` : Instantiates all 6 sub-screens (`Dashboard_Screen` , `CustomerManagement_Screen` , `ServiceQueue_Screen` , `Billing_Screen` , `Appointment_Screen` , `ServiceManagement_Screen`), extracts their internal panels via getter methods, and inserts them into `jTabbedPane1` !
    - `setupControls()` & `setupTable()` : Configures vehicle search bar, action column renderer, and pagination.
    - `loadVehicles()` : Loads active cars into memory.

> **Evaluator Defense Tip:** If your evaluator asks *"Where does the application start and how does it navigate?"*, answer: *"The application begins in LoginForm.main(), which initializes FlatLaf modern UI. When credentials are validated by Login_BLL and Login_DAO, LoginForm closes itself and launches VehicleManagement_Screen. VehicleManagement_Screen is our master container window that embeds the main panels of all other screens into a JTabbedPane."*

2. Database Schema, Relationships & Foreign Keys

The database is named `carcare_db` running on MySQL via XAMPP. After our system audit and cleanup, there are exactly **9 active, fully-connected tables**. All unused tables (`payments` , `service_record_items`) and duplicate indexes were cleanly removed.

Table Name	Rows	Primary Key	Outbound Foreign Keys	Role in System
`users`	2	`user_id`	None	Stores administrator login credentials (email & password).
`customers`	6	`customer_id`	None	Master directory of vehicle owners (name, phone, email, address).
`vehicles`	7	`vehicle_id`	`customer_id → customers`	Registered vehicles with unique registration number, make, model, year, mileage.
`mechanics`	4	`mechanic_id`	None	Workshop technicians with specialization and availability status.
`services`	5	`service_id`	None	Workshop service catalog (e.g. Oil Change, Brake Inspection, Wheel Alignment).
`appointments`	3	`appointment_id`	`customer_id → customers` `vehicle_id → vehicles` `mechanic_id → mechanics`	Future bookings with date, time, status ('Pending', 'Confirmed', 'Completed').
`service_records`	2	`record_id`	`vehicle_id → vehicles` `mechanic_id → mechanics`	Core work order table binding cars, assigned mechanics, date, service description.
`service_queue`	2	`queue_id`	`vehicle_id → vehicles` `record_id → service_records`	Live FIFO workshop bay queue with queue position, start time, completion time.
`bills`	2	`bill_id`	`record_id → service_records` (UNIQUE 1:1)	Customer invoices with service total, parts, discount, grand total, status ('Paid', 'Unpaid').

Referential Integrity & Cascade Rules

- **Customer Deletion Cascade:** `vehicles.customer_id → customers.customer_id ON DELETE CASCADE` . If a customer is deleted, all their registered vehicles and appointments are automatically removed by MySQL engine.
- **Vehicle Deletion Cascade:** `service_records.vehicle_id → vehicles.vehicle_id ON DELETE CASCADE` and `service_queue.vehicle_id ON DELETE CASCADE` . Deleting a car cascades to queue items and work logs.
- **Mechanic Safety:** `service_records.mechanic_id → mechanics.mechanic_id ON DELETE SET NULL` . If a mechanic profile is deleted, historical service records are preserved with `mechanic_id = NULL` rather than destroyed.
- **Billing 1:1 Unique Constraint:** `bills.record_id` has a `UNIQUE` constraint, guaranteeing that each completed service job has at most ONE invoice.

Audit Note: Why were `payments` and `service_record_items` removed?
During the audit, we found that `payments` had 0 rows because the application updates `bills.status = 'Paid'` directly instead of writing separate payment ledger slips. Similarly, `service_record_items` had 0 rows because work orders store service names directly in `service_records.description`. Removing these 2 unused tables simplified the database and prevents evaluator penalties for dead schema.

3. How the Tabbed Navigation Works (Step-by-Step)

One of the most common evaluator questions is: *"How does your tabbed interface work? Are these separate frames or embedded panels?"*

The Master Container Pattern

The entire multi-tab workshop application runs inside **one single JFrame**: `VehicleManagement_Screen`. Inside this frame is a `JTabbedPane` named `jTabbedPane1`.

Instead of opening and closing multiple separate windows, `VehicleManagement_Screen` creates an instance of each screen and embeds that screen's main content panel into the tab bar. Here is the exact code from `VehicleManagement_Screen.java` (Lines 90-113):

```
// 1. Dashboard Tab (Inserted at index 0)
dashboardScreen = new Dashboard_Screen();
jTabbedPane1.insertTab("  Dashboard", null, dashboardScreen.getDashboardPanel(), null, 0);

// 2. Vehicles Tab (Uses the panel built into VehicleManagement_Screen itself)
// It sits at index 1 by default.

// 3. Customers Tab (Index 2)
customerScreen = new CustomerManagement_Screen();
jTabbedPane1.addTab("  Customers", customerScreen.getCustomerPanel());

// 4. Service Queue Tab (Index 3)
queueScreen = new ServiceQueue_Screen();
jTabbedPane1.addTab("  Service Queue", queueScreen.getServiceQueuePanel());

// 5. Billing Tab (Index 4)
billingScreen = new Billing_Screen();
jTabbedPane1.addTab("  Billing", billingScreen.getBillingPanel());

// 6. Appointments Tab (Index 5)
appointmentScreen = new Appointment_Screen();
jTabbedPane1.addTab("  Appointments", appointmentScreen.getAppointmentsPanel());

// 7. Services Tab (Index 6)
serviceScreen = new ServiceManagement_Screen();
jTabbedPane1.addTab("  Services", serviceScreen.getServicePanel());
```

How Each Screen Exposes Its Content Panel

Each individual screen class implements a public getter that returns its internal content `JPanel`:

- `Dashboard_Screen.java` (Line 70): `public javax.swing.JPanel getDashboardPanel() { return jPanel1; }`
- `CustomerManagement_Screen.java`: `public javax.swing.JPanel getCustomerPanel() { return jPanel1; }`
- `ServiceQueue_Screen.java` (Line 662): `public javax.swing.JPanel getServiceQueuePanel() { return jPanel1; }`
- `Billing_Screen.java` (Line 581): `public javax.swing.JPanel getBillingPanel() { return jPanel1; }`
- `Appointment_Screen.java`: `public javax.swing.JPanel getAppointmentsPanel() { return jPanel1; }`
- `ServiceManagement_Screen.java`: `public javax.swing.JPanel getServicePanel() { return jPanel1; }`

Automatic Tab-Switch Data Refresh Listener

When the user clicks on any tab, an event listener detects the tab index change and immediately re-queries the database so the data is always 100% fresh! Look at lines 125-143 of `VehicleManagement_Screen.java`:

```
jTabbedPane1.addChangeListener(e -> {
    int idx = jTabbedPane1.getSelectedIndex();
    if (idx == 0 && dashboardScreen != null) {
        dashboardScreen.loadDashboardStats(); // Refresh metric counts & activity
    } else if (idx == 1) {
        loadVehicles(); // Refresh vehicles table
    } else if (idx == 2 && customerScreen != null) {
        customerScreen.loadCustomers(); // Refresh customers table
    } else if (idx == 3 && queueScreen != null) {
        queueScreen.loadQueueData(); // Refresh FIFO queue & active bay
    } else if (idx == 4 && billingScreen != null) {
        billingScreen.loadBills(); // Auto-create & fetch invoices
    } else if (idx == 5 && appointmentScreen != null) {
        appointmentScreen.loadAppointments(); // Refresh appointment calendar
    } else if (idx == 6 && serviceScreen != null) {
        serviceScreen.loadServices(); // Refresh catalog & vehicle combos
        serviceScreen.loadVehicles();
    }
});
```

> **Key Takeaway:** Switching tabs is **reactive and live**. If you add a car in the Vehicles tab and switch to the Dashboard tab, the "Total Vehicles" metric immediately updates without needing to restart the application!

4. Data Access Layer (DAO) — Every Function & SQL Query

All database queries reside exclusively in the `DataAccessLayer` package. Every function uses parameterized `PreparedStatement` to prevent SQL Injection.

4.1 DB_Connection.java — Database Connection Factory

Purpose: Establishes a JDBC physical connection to MySQL.

```
public class DB_Connection {
    private static final String URL = "jdbc:mysql://localhost:3306/carcare_db";
    private static final String USER = "root";
    private static final String PASSWORD = "";

    public Connection getConnection() {
        try {
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (SQLException e) {
            System.out.println("Database connection failed!");
            e.printStackTrace();
            return null;
        }
    }
}
```

4.2 Login_DAO.java

Function: `validateLogin(String email, String password)` • Returns: `boolean`

SQL Query: `SELECT * FROM users WHERE email = ? AND password = ?`

What it does: Binds email to parameter 1, password to parameter 2. Executes query. If `rs.next()` finds a match, returns `true`.

4.3 Customer_DAO.java

Method Signature	Exact SQL Query Executed	Operation & Return
`addCustomer(Customer c)`	`INSERT INTO customers (full_name, phone, email, address) VALUES (?, ?, ?, ?)`	Executes insert with `RETURN_GENERATED_KEYS`. Returns auto-generated `customer_id` (or -1).
`getAllCustomers()`	`SELECT * FROM customers ORDER BY customer_id DESC`	Loops `ResultSet`, instantiates `Customer` objects, returns `List<Customer>`.
`updateCustomer(Customer c)`	`UPDATE customers SET full_name=?, phone=?, email=?, address=? WHERE customer_id=?`	Updates profile details matching `customer_id`. Returns `boolean`.
`deleteCustomer(int id)`	`DELETE FROM customers WHERE customer_id=?`	Deletes customer. Cascades to vehicles & bookings. Returns `boolean`.

4.4 Vehicle_DAO.java

Method Signature	Exact SQL Query Executed	Operation & Return
`getAllVehicles()`	`SELECT v.*, c.full_name AS customer_name FROM vehicles v LEFT JOIN customers c ON v.customer_id = c.customer_id ORDER BY v.vehicle_id DESC`	Joins vehicles with customers to display owner name. Returns `List<Vehicle>`.
`addVehicle(Vehicle v)`	`INSERT INTO vehicles (customer_id, registration_no, make, model, vehicle_year, mileage) VALUES (?, ?, ?, ?, ?, ?)`	Binds customer ID and car specs. Returns `boolean`.
`deleteVehicle(String regNo)`	`DELETE FROM vehicles WHERE registration_no = ?`	Deletes vehicle by registration plate. Cascades to records/queue. Returns `boolean`.
`updateVehicle(regNo, make, model, year, mileage)`	`UPDATE vehicles SET make=?, model=?, vehicle_year=?, mileage=? WHERE registration_no=?`	Updates vehicle specifications by registration plate. Returns `boolean`.
`getVehicleByRegNo(String regNo)`	`SELECT v.*, c.full_name AS customer_name FROM vehicles v LEFT JOIN customers c ON v.customer_id = c.customer_id WHERE v.registration_no = ?`	Used by Verify buttons in queue & appointment dialogs. Returns `Vehicle` (or null).

4.5 Service_DAO.java

Method Signature	Exact SQL Query Executed	Operation & Return
`getAllServices()`	`SELECT * FROM services ORDER BY service_id ASC`	Fetches active service catalog. Returns `List<Service>`.
`addService(name, cost, time, desc)`	`INSERT INTO services (service_name, description, cost, estimated_time, status) VALUES (?, ?, ?, ?, 'Active')`	Inserts newly created service into catalog. Returns `boolean`.
`updateService(id, name, cost, time)`	`UPDATE services SET service_name=?, cost=?, estimated_time=? WHERE service_id=?`	Modifies price or duration of existing service. Returns `boolean`.
`deleteService(int id)`	`DELETE FROM services WHERE service_id = ?`	Deletes service entry from catalog. Returns `boolean`.
`assignServiceToVehicle(vehId, name)`	Delegates to `ServiceQueue_DAO.addToQueue(vehId, name, "Ahmed")`	Dispatches car to live queue with assigned technician. Returns `boolean`.

4.6 ServiceQueue_DAO.java — Workshop Queue Engine

Manages FIFO queue ordering and active workshop bay transitions.

Method Signature	SQL Queries & Mechanics	Operation & Return
`addToQueue(vehicleId, serviceName, mechanicName)`	1. `SELECT mechanic_id FROM mechanics WHERE full_name LIKE ? LIMIT 1` 2. `INSERT INTO service_records (vehicle_id, mechanic_id, service_date, description, status) VALUES (?, ?, CURDATE(), ?, 'Pending')` 3. `SELECT COALESCE(MAX(queue_position), 0) + 1 FROM service_queue WHERE status = 'Waiting'` 4. `INSERT INTO service_queue (vehicle_id, record_id, queue_position, status) VALUES (?, ?, ?, 'Waiting')`	Atomic SQL transaction with rollback. Creates work log and queue ticket. Returns `boolean`.
`getWaitingQueue()`	5-Table JOIN: `service_queue sq JOIN vehicles v ON sq.vehicle_id = v.vehicle_id LEFT JOIN customers c ... LEFT JOIN service_records sr ... LEFT JOIN mechanics m ... WHERE sq.status = 'Waiting' ORDER BY sq.queue_position ASC, sq.added_at ASC`	Constructs ordered FIFO queue. Returns `Queue<ServiceTicket>` (Java `LinkedList`).
`getActiveService()`	Same 5-table JOIN with `WHERE sq.status = 'In Service' ORDER BY sq.started_at DESC LIMIT 1`	Fetches vehicle currently inside Bay 1. Returns `ServiceTicket`.
`callNextVehicle()`	1. Polls first ticket from `getWaitingQueue()`. 2. `UPDATE service_queue SET status = 'In Service', started_at = NOW() WHERE queue_id = ?` 3. `UPDATE service_records SET status = 'In Progress' WHERE record_id = ?` 4. `UPDATE service_queue SET queue_position = queue_position - 1 WHERE status = 'Waiting'`	Moves first waiting car into active bay and decrements waiting positions. Returns `ServiceTicket`.
`markAsCompleted(queueId, recordId)`	1. `UPDATE service_queue SET status = 'Completed', completed_at = NOW() WHERE queue_id = ?` 2. `UPDATE service_records SET status = 'Completed' WHERE record_id = ?`	Empties active bay, stamps completion timestamp. Returns `boolean`.
`removeFromQueue(int queueId)`	`UPDATE service_queue SET status = 'Cancelled' WHERE queue_id = ? AND status = 'Waiting'` followed by `resequenceQueuePositions()`	Cancels ticket and re-numbers positions (1, 2, 3...). Returns `boolean`.
`getWaitingCount()`	`SELECT COUNT(*) FROM service_queue WHERE status = 'Waiting'`	Returns `int` count for Dashboard card.
`getUnderServiceCount()`	`SELECT COUNT(*) FROM service_queue WHERE status = 'In Service'`	Returns `int` count for Dashboard card.
`getCompletedCount()`	`SELECT COUNT(*) FROM service_queue WHERE status = 'Completed'`	Returns `int` total completed cars.

`getRecentQueueTickets(int limit)`	5-table JOIN with `ORDER BY sq.queue_id DESC LIMIT ?`	Returns `List<ServiceTicket>` for Recent Activity table.

4.7 Billing_DAO.java — Invoicing Engine

Method Signature	Exact SQL Query Executed	Operation & Return
`getAllBills()`	1. Calls `autoCreateBills()` first. 2. `SELECT b.bill_id, v.registration_no, CONCAT(v.make,' ',v.model) AS vehicle_name, COALESCE(c.full_name,'Customer') AS customer_name, COALESCE(sr.description,'General Service') AS service_name, b.grand_total, b.status FROM bills b LEFT JOIN service_records sr ON b.record_id = sr.record_id LEFT JOIN vehicles v ON sr.vehicle_id = v.vehicle_id LEFT JOIN customers c ON v.customer_id = c.customer_id ORDER BY b.bill_id DESC`	Fetches all customer invoices with car and owner details. Returns `List<Bill>`.
`autoCreateBills()` (Private)	`INSERT INTO bills (record_id, bill_date, service_total, grand_total, status) SELECT sr.record_id, CURDATE(), 5000, 5000, 'Unpaid' FROM service_records sr LEFT JOIN bills b ON sr.record_id = b.record_id WHERE b.bill_id IS NULL AND sr.status = 'Completed'`	Auto-generates missing invoice records for any completed jobs. Returns `void`.
`markBillPaid(int billId)`	`UPDATE bills SET status = 'Paid' WHERE bill_id = ?`	Marks invoice as Paid. Returns `boolean`.
`updateBillTotal(int billId, double newTotal)`	`UPDATE bills SET grand_total = ? WHERE bill_id = ?`	Updates grand total when adding extra parts. Returns `boolean`.

4.8 Appointment_DAO.java

Method Signature	Exact SQL Query Executed	Operation & Return
`getAllAppointments()`	`SELECT a.appointment_id, a.appointment_date, a.appointment_time, v.registration_no, CONCAT(v.make, ' ', v.model) AS vehicle_name, COALESCE(c.full_name, 'Customer') AS customer_name, COALESCE(a.notes, 'General Service') AS service_name, a.status FROM appointments a LEFT JOIN vehicles v ON a.vehicle_id = v.vehicle_id LEFT JOIN customers c ON a.customer_id = c.customer_id WHERE a.status != 'Completed' ORDER BY a.appointment_id DESC`	Loads pending/confirmed bookings. Returns `List<Appointment>`.
`addAppointment(...)`	`INSERT INTO appointments (customer_id, vehicle_id, appointment_date, appointment_time, status, notes) VALUES (?, ?, ?, ?, 'Confirmed', ?)`	Schedules booking. Returns `boolean`.

`confirmAppointment(id)`	`UPDATE appointments SET status = 'Confirmed' WHERE appointment_id = ?`	Confirms pending booking. Returns `boolean`.
`checkInToQueue(appId, regNo)`	1. `UPDATE appointments SET status = 'Completed' WHERE appointment_id = ?` 2. `Vehicle_DAO.getVehicleByRegNo(regNo)` 3. `ServiceQueue_DAO.addToQueue(vehicleId, "General Service / Oil Change", "Ahmed")`	Marks booking done and injects car into live queue! Returns `boolean`.

5. Business Logic Layer (BLL) & Domain Entities

The `BusinessLayer` package encapsulates domain entities (POJOs) and business validation services.

5.1 Business Services

1. Login_BLL.java

```
public class Login_BLL {
    private final Login_DAO loginDAO;

    public Login_BLL(Login_DAO loginDAO) {
        this.loginDAO = loginDAO;
    }

    public boolean login(String email, String password) {
        // Business Rule: Null or empty credentials are rejected before hitting DB
        if (email == null || email.trim().isEmpty()) return false;
        if (password == null || password.trim().isEmpty()) return false;
        return loginDAO.validateLogin(email.trim(), password);
    }
}
```

2. Vehicle_BLL.java

```
public class Vehicle_BLL {
    private Vehicle_DAO vehicleDAO = new Vehicle_DAO();

    public List<Vehicle> getAllVehicles() {
        return vehicleDAO.getAllVehicles();
    }

    public boolean addVehicle(Vehicle vehicle) {
        // Business Rules:
        if (vehicle == null) return false;
        if (vehicle.getRegistrationNo() == null || vehicle.getRegistrationNo().trim().isEmpty()) retu
        if (vehicle.getVehicleYear() < 1900 || vehicle.getVehicleYear() > 2030) return false;
        if (vehicle.getMileage() < 0) return false;
        return vehicleDAO.addVehicle(vehicle);
    }

    public boolean deleteVehicle(String registrationNo) {
        if (registrationNo == null || registrationNo.trim().isEmpty()) return false;
        return vehicleDAO.deleteVehicle(registrationNo.trim());
    }
}
```

3. Customer_BLL.java

```
public class Customer_BLL {
    private Customer_DAO dao = new Customer_DAO();

    public List<Customer> getAllCustomers() {
        return dao.getAllCustomers();
    }
```

```
    public boolean updateCustomer(Customer c) {
        return dao.updateCustomer(c);
    }

    public boolean deleteCustomer(int id) {
        return dao.deleteCustomer(id);
    }
}
```

5.2 Domain Entities & DTOs

Entity Class	Fields Mapped	Constructors	Purpose
`Customer.java`	`customerId`, `fullName`, `phone`, `email`, `address`	Default, 4-param (new), 5-param (existing with ID)	Encapsulates vehicle owner details.
`Vehicle.java`	`vehicleId`, `customerId`, `customerName`, `registrationNo`, `make`, `model`, `vehicleYear`, `mileage`	Default, 6-param (new), 8-param (with joined customerName)	Encapsulates registered car data.
`Service.java`	`serviceId`, `serviceName`, `description`, `cost`, `estimatedTime`, `status`	Default, 6-param	Represents service offering in workshop catalog.
`ServiceTicket.java`	`queueId`, `vehicleId`, `recordId`, `queuePosition`, `registrationNo`, `make`, `model`, `vehicleYear`, `customerName`, `serviceName`, `mechanicName`, `addedAt`, `startedAt`, `completedAt`, `status`	Default, 15-param constructor	View Model for FIFO queue cards & activity table. Includes `getVehicleFullName()`.
`Appointment.java`	`appId`, `date`, `time`, `regNo`, `vehicle`, `customer`, `service`, `status`	8-param constructor	View Model for appointment scheduling table.
`Bill.java`	`billId`, `regNo`, `vehicle`, `customer`, `service`, `total`, `status`	7-param constructor	View Model for billing table and receipt printing.

6. Presentation Layer — Screen-by-Screen Code Walkthrough

Every screen in CarCare uses **FlatLaf Light** theme with modern rounded cards (`arc: 25`), pill buttons (`roundRect`), and customized table renderers.

6.1 LoginForm.java — Entry Screen

- **Design & Layout:** Uses `BackgroundPanel` drawing `assets/full.jpeg` . Inside is `jPanel1` , an overridden panel that draws a semi-translucent slate card (`rgba(15, 23, 42, 0.92)`) with 24px rounded corners and an anti-aliased border.
- **Fullscreen Centering (The Fix):** `backgroundPanelLayout.setVerticalGroup` uses `.addContainerGap(75, Short.MAX_VALUE)` on both the top and bottom. This acts as dual springs, keeping the login card centered vertically and horizontally at all screen sizes (1024×640, 1440×900, 1080p, 4K).
- **Password Security:** Uses `JPasswordField` with `FlatLaf.style: showRevealButton: true` (native eye toggle).
- **Login Handler (Lines 39-63):**

```
private void handleLogin() {
    String email = jTextField1.getText().trim();
    String password = new String(jPasswordField1.getPassword()).trim();

    if (email.isEmpty() || password.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Please enter both email and password.", "Validatio
        return;
    }

    if (loginBLL.login(email, password)) {
        this.dispose(); // Close login
        new VehicleManagement_Screen().setVisible(true); // Launch master screen
    } else {
        JOptionPane.showMessageDialog(this, "Invalid email or password.", "Login Failed", JOpti
    }
}
```

6.2 Dashboard_Screen.java — Metrics & Live Activity

- **6 Metric Cards:**
  1. **Total Customers:** Indigo card • `customerDAO.getAllCustomers().size()`
  2. **Total Vehicles:** Car Blue card • `vehicleDAO.getAllVehicles().size()`
  3. **Waiting in Queue:** Amber Gold card • `queueDAO.getWaitingQueue().size()`
  4. **Currently in Service:** Purple card • `queueDAO.getUnderServiceCount()`
  5. **Completed Services:** Teal card • `queueDAO.getCompletedCount()`
  6. **Total Revenue:** Green card • Sum of `b.getTotal()` across all bills
- **Quick Operations Buttons:**
  - `btnRegister` ("+ Add Vehicle"): `new AddVehicle_SCREEN(this, true).setVisible(true); loadDashboardData();`
  - `btnAppointment` ("New Booking"): `new NewBooking_SCREEN(this, true).setVisible(true); loadDashboardData();`
  - `btnInvoice` ("View Billing"): `new Billing_Screen().setVisible(true);`
- **Recent Activity Table:** Alternating zebra rows (white / #f8fafc), colored status labels (Amber for Waiting, Green for Completed, Purple for In Service). Populated by `queueDAO.getRecentQueueTickets(15)` .

6.3 VehicleManagement_Screen.java — Master Window & Vehicle Tab

- **Master Tab Container:** Sets up top tab bar (`setupTabs()` , 42px height, navy blue active indicator).
- **Live Search Filter (Lines 163-176):** `jTextField1.getDocument().addDocumentListener(...)` fires on every keystroke, calling `filterVehicles(text)` to filter rows by Registration No., Make, Model, or Customer Name in real time.
- **Add Vehicle Button:** Opens `AddVehicle_SCREEN` modal dialog. Refreshes table on close.
- **Interactive Action Column (Edit / Delete Icons):** Table uses a custom cell renderer drawing `assets/edit.png` and `assets/trash.png` . A `MouseListener` on `jTable1` detects click coordinates:
  - If clicked on left half of Action column (Edit): Opens `EditVehicle_SCREEN` and passes selected `Vehicle` object.
  - If clicked on right half (Delete): Prompts `JOptionPane.showConfirmDialog` . If confirmed, executes `vehicleDAO.deleteVehicle(regNo)` .
- **Adaptive Pagination:** Dynamically calculates page size based on window height (`pageSize = max(5, (tableHeight - 40) / 40)`) with Next/Previous buttons.

6.4 CustomerManagement_Screen.java — Customer Directory

- **Purpose:** Manages customer contact information and vehicle ownership.
- **Table Loading:** `customerBLL.getAllCustomers()` • Populates Customer ID, Name, Phone, Email, Address.
- **Live Search:** Filters customer list dynamically across name, phone, email, and address.
- **Edit & Delete:** Table row click allows editing via `EditCustomer_SCREEN` or deleting via `customerBLL.deleteCustomer(id)` with cascade warning.

6.5 ServiceQueue_Screen.java — Workshop Queue & Live Timer

- **Active Service Bay (Bay 1 Card):** Displays vehicle currently undergoing repair (Reg No badge, customer, service, mechanic). Includes a live animated timer (`javax.swing.Timer`) ticking every second to show elapsed time since `started_at` !
- **FIFO Queue Table:** Displays upcoming cars ordered by `queue_position ASC` .
- **Call Next Vehicle Button (`jButton5`):** Calls `queueDAO.callNextVehicle()` . The first waiting vehicle is automatically moved to the active bay, and remaining queue positions decrement by 1.
- **Mark Completed Button (`jButton2`):** Calls `queueDAO.markAsCompleted(queueId, recordId)` . Empties Bay 1 and records completion time.
- **Remove from Queue Button (`jButton7`):** Cancels selected car and re-sequences remaining positions (1, 2, 3...).
- **Add to Queue Button (`jButton1`):** Opens `AddtoQueue_SCREEN` modal dialog.

6.6 Appointment_Screen.java — Booking Calendar

- **Filter Buttons:** "All", "Confirmed", and "Pending" buttons toggle local list display.
- **Book Appointment Button (`jButton1`):** Opens `NewBooking_SCREEN` modal dialog.
- **Check-In Button (`jButton2`):** Select an appointment, click Check-In. Executes `appointmentDAO.checkInToQueue(appId, regNo)` : marks appointment Completed and immediately inserts the vehicle into the live workshop queue!
- **Confirm Booking Button (`jButton5`):** Marks a 'Pending' appointment as 'Confirmed' via `appointmentDAO.confirmAppointment(appId)` .

6.7 ServiceManagement_Screen.java — Catalog & Assignment

- **Vehicle Dropdown (`jComboBox1`):** Loaded from `vehicleDAO.getAllVehicles()` .
- **Service Dropdown (`jComboBox2`):** Loaded from `serviceDAO.getAllServices()` . Changing selection updates cost and time display labels in real time.
- **+ Add Service Button (`jButton1`):** Shows 3 sequential `JOptionPane.showInputDialog` pop-ups for Name, Cost, and Time, then calls `serviceDAO.addService()` .
- **Assign to Vehicle Button (`jButton2`):** Takes selected car and service, calls `serviceDAO.assignServiceToVehicle()` , which puts the car into the service queue!

6.8 Modal Dialog Screens (JDialog)

Dialog Class	Triggered From	Mechanics & Database Operation
`AddVehicle_SCREEN`	Dashboard & Vehicle Tab ("+ Add Vehicle")	Dual-entity creation. Validates 11-digit phone. Saves Customer first via `Customer_DAO.addCustomer()` to obtain auto-generated ID, then saves Vehicle via `Vehicle_DAO.addVehicle()` with that Customer ID.

`EditVehicle_SCREEN`	Vehicle Table (Edit icon)	Receives `Vehicle` object via `setVehicleData()`. Locks customer name. On Save: calls `Vehicle_DAO.updateVehicle()`.
`AddtoQueue_SCREEN`	Service Queue ("+ Add to Queue")	User inputs Reg No. **"Verify"** button calls `vehicleDAO.getVehicleByRegNo()` • turns badge green if found. Select service & mechanic • calls `serviceQueueDAO.addToQueue()`.
`NewBooking_SCREEN`	Appointments ("Book Appointment")	Verifies vehicle via registration number. Inputs date, time, service. Calls `appointmentDAO.addAppointment()`.
`EditCustomer_SCREEN`	Customer Table (Edit click)	Receives `Customer` object. On Save: calls `Customer_DAO.updateCustomer()`.

7. Every Single Button Click Traced to Database (15 Flows)

Here is the exact trace for every single button in the CarCare system from the user click down to the SQL query:

#	Screen	Button Name	Java Event Handler Method	Exact SQL Query Executed
1	`LoginForm`	**LOGIN**	`handleLogin()` → `Login_BLL.login()`	`SELECT * FROM users WHERE email = ? AND password = ?`
2	`Dashboard_Screen`	**+ Add Vehicle**	`btnRegisterActionPerformed()`	Opens `AddVehicle_SCREEN` dialog (Flow #4 below).
3	`Dashboard_Screen`	**New Booking**	`btnAppointmentActionPerformed()`	Opens `NewBooking_SCREEN` dialog (Flow #11 below).
4	`AddVehicle_SCREEN`	**+ Register & Save**	`registerAndSave()`	1. `INSERT INTO customers (full_name, phone, email, address) VALUES (?, ?, ?, ?)` 2. `INSERT INTO vehicles (customer_id, registration_no, make, model, vehicle_year, mileage) VALUES (?, ?, ?, ?, ?, ?)`
5	`VehicleManagement`	**Delete (Table icon)**	`jTable1MouseAdapter` → `deleteVehicle()`	`DELETE FROM vehicles WHERE registration_no = ?` (Cascades)
6	`EditVehicle_SCREEN`	**Save Changes**	`jButton1ActionPerformed()`	`UPDATE vehicles SET make=?, model=?, vehicle_year=?, mileage=? WHERE registration_no=?`
7	`AddtoQueue_SCREEN`	**Verify**	`btnVerifyActionPerformed()`	`SELECT v.*, c.full_name FROM vehicles v LEFT JOIN customers c ON v.customer_id = c.customer_id WHERE v.registration_no = ?`
8	`AddtoQueue_SCREEN`	**Confirm & Add**	`btnConfirmActionPerformed()`	1. `SELECT mechanic_id FROM mechanics WHERE full_name LIKE ?` 2. `INSERT INTO service_records (vehicle_id, mechanic_id, service_date, description, status) VALUES (?, ?, CURDATE(), ?, 'Pending')` 3. `INSERT INTO service_queue (vehicle_id, record_id,`

				`queue_position, status) VALUES (?, ?, ?, 'Waiting')`
9	`ServiceQueue_Screen`	**Call Next Vehicle**	`callNextVehicle()`	1. `UPDATE service_queue SET status = 'In Service', started_at = NOW() WHERE queue_id = ?` 2. `UPDATE service_records SET status = 'In Progress' WHERE record_id = ?` 3. `UPDATE service_queue SET queue_position = queue_position - 1 WHERE status = 'Waiting'`
10	`ServiceQueue_Screen`	**Mark Completed**	`markActiveServiceCompleted()`	1. `UPDATE service_queue SET status = 'Completed', completed_at = NOW() WHERE queue_id = ?` 2. `UPDATE service_records SET status = 'Completed' WHERE record_id = ?`
11	`NewBooking_SCREEN`	**Confirm Booking**	`btnConfirmActionPerformed()`	`INSERT INTO appointments (customer_id, vehicle_id, appointment_date, appointment_time, status, notes) VALUES (?, ?, ?, ?, 'Confirmed', ?)`
12	`Appointment_Screen`	**Check-In**	`checkInToQueue()`	1. `UPDATE appointments SET status = 'Completed' WHERE appointment_id = ?` 2. Pushes vehicle into `service_records` and `service_queue` (Flow #8).
13	`ServiceManagement`	**+ Add Service**	`jButton1ActionPerformed()`	`INSERT INTO services (service_name, description, cost, estimated_time, status) VALUES (?, ?, ?, ?, 'Active')`
14	`Billing_Screen`	**+ Add Extra Part**	`addExtraPart()`	`UPDATE bills SET grand_total = ? WHERE bill_id = ?`
15	`Billing_Screen`	**Collect Payment**	`collectPayment()`	`UPDATE bills SET status = 'Paid' WHERE bill_id = ?`

8. In-Depth Billing & Invoicing Mechanics

The Billing module (`Billing_Screen.java`) is one of the most sophisticated modules in CarCare. Here is exactly how it functions:

8.1 Automatic Invoice Generation

When the user clicks the Billing tab, `Billing_DAO.getAllBills()` executes. Before returning the list, it calls `autoCreateBills()` (Lines 58-75 in `Billing_DAO.java`):

```
INSERT INTO bills (record_id, bill_date, service_total, grand_total, status)
SELECT sr.record_id, CURDATE(), 5000, 5000, 'Unpaid'
FROM service_records sr
LEFT JOIN bills b ON sr.record_id = b.record_id
WHERE b.bill_id IS NULL AND sr.status = 'Completed'
```

What this SQL query does: It finds every service job in `service_records` that has `status = 'Completed'` but does not yet have a matching invoice in `bills` . It automatically creates an invoice for today's date with `status = 'Unpaid'` . Because of the `UNIQUE KEY (record_id)` constraint, duplicate bills are impossible!

8.2 The Invoice Detail Card

When an invoice row is clicked in `jTable1` , `showBillDetails(Bill b)` (Lines 492-519) populates the right-hand panel (`jPanel3`):

- **Header:** Displays Invoice number (`#INV-101`), customer name, vehicle make/model, and license plate.
- **Itemized Breakdown:**
  - Service package base cost (e.g. Rs. 5,000)
  - Extra Parts / Addons (if total > 5,000, calculated as `total - 5000`)
  - Diagnostics & Inspection (included, highlighted in green)
  - Workshop Labor Charges (included, highlighted in green)
- **Subtotal & Tax:** Displays subtotal and tax.
- **Payment State:** If status is 'Paid', the "Collect Payment" button is automatically disabled and labeled "Paid". If 'Unpaid', the button is enabled in blue.

8.3 The Dynamic Discount Engine

When the user changes the discount dropdown (`jComboBox1`), `applyDiscount()` (Lines 522-532) fires:

```
private void applyDiscount() {
    if (selectedBill == null) return;
    int idx = jComboBox1.getSelectedIndex();
    double discountPct = (idx == 1) ? 0.05 : (idx == 2) ? 0.10 : 0.0;
    double discountAmt = selectedBill.getTotal() * discountPct;
    double grandTotal = selectedBill.getTotal() - discountAmt;

    jLabel10.setText("-Rs. " + String.format("%.0f", discountAmt));
    jLabel13.setText("Grand Total: Rs. " + String.format("%.0f", grandTotal));
}
```

- Index 0: "No Discount (0%)" → Grand Total = Full Total
- Index 1: "Loyalty Discount (5%)" → 5% deducted
- Index 2: "Corporate Fleet (10%)" → 10% deducted

8.4 Adding Extra Parts to Bill

When the "+ Add Extra Part" button (`jButton1`) is clicked, `addExtraPart()` prompts `JOptionPane.showInputDialog("Enter Part Price (Rs):")` . If the user enters `1500` :

1. It calculates `newTotal = selectedBill.getTotal() + 1500` .
2. It calls `billingDAO.updateBillTotal(selectedBill.getBillId(), newTotal)` .
3. Executes: `UPDATE bills SET grand_total = ? WHERE bill_id = ?` .
4. Reloads the bills list, and the itemized card automatically shows: *"Extra Parts / Addons: Rs. 1,500"*!

8.5 Printing the Official Receipt

When "Print Receipt" (`jButton5`) is clicked, `printReceipt()` (Lines 560-579) determines payment method (Cash / Card radio buttons) and constructs a monospaced ASCII receipt displayed inside a read-only `JTextArea` modal dialog:

```
==================================
       CARCARE AUTO SERVICES
          Customer Receipt
==================================
Invoice:  #INV-2
Customer: Sarah Connor
Vehicle:  Honda Civic (KHI-5678)
Service:  Brake Pad Replacement
Method:   Cash
Total:    Rs. 6,500
Status:   Paid
==================================
      Thank you for your visit!
==================================
```

9. Evaluator Defense Q&A (Questions & Exact Answers)

Here are the exact questions your evaluator is guaranteed or likely to ask, along with the precise, professional answers you should give:

Q1: What architecture does this application follow and why did you choose it?
Your Answer: *"CarCare follows the 3-Tier Architecture: Presentation Layer, Business Layer, and Data Access Layer. I chose this to decouple UI components from database logic. The UI handles user input and styling, the Business Layer enforces validation rules, and the Data Access Layer executes SQL queries. This makes the system maintainable, testable, and prevents database code from leaking into GUI forms."*

Q2: How is the database connected and how do you prevent SQL injection?
Your Answer: *"The database is connected via JDBC using DB_Connection.getConnection(), which connects to MySQL on localhost:3306. To completely prevent SQL injection, every single query in our DAOs uses java.sql.PreparedStatement with '?' placeholders. User input is never concatenated directly into SQL strings."*

Q3: How does the application transition from Login to the main Dashboard?
Your Answer: *"When the user clicks Login, LoginForm.handleLogin() validates the credentials via Login_BLL and Login_DAO. Upon successful authentication, LoginForm calls this.dispose() to free memory and instantiates new VehicleManagement_Screen().setVisible(true). VehicleManagement_Screen is our master window that hosts all the workshop tabs."*

Q4: How do the tabs work in your system? Are they separate windows?
Your Answer: *"No, they are not separate windows. VehicleManagement_Screen contains a JTabbedPane. In setupTabs(), it instantiates each screen class (Dashboard_Screen, CustomerManagement_Screen, ServiceQueue_Screen, Billing_Screen, etc.) and extracts their main content panels using getter methods like getDashboardPanel() or getBillingPanel(). It embeds these panels into the tab bar. When the user switches tabs, a ChangeListener automatically refreshes the data for that tab from MySQL."*

Q5: How does the FIFO service queue work?
Your Answer: *"The service queue implements a First-In, First-Out (FIFO) structure. When a car is added to the queue via addToQueue(), it is assigned the next queue_position (MAX + 1). ServiceQueue_DAO.getWaitingQueue() returns a Queue<ServiceTicket> backed by a Java LinkedList. When the workshop manager clicks 'Call Next Vehicle', callNextVehicle() polls the head of the queue, updates its status to 'In Service' in Bay 1, and decrements all other waiting queue positions by 1."*

Q6: What happens when you register a vehicle? Explain the database flow.
Your Answer: *"When the user clicks 'Register & Save' in AddVehicle_SCREEN, it performs a dual-entity transaction: first, it saves the customer via Customer_DAO.addCustomer() using RETURN_GENERATED_KEYS to obtain the auto-increment customer_id. Then, it creates the Vehicle object using that customer_id and saves it via Vehicle_DAO.addVehicle(). This maintains strict referential integrity."*

Q7: How does billing work when a service is finished?
Your Answer: *"When a service is marked as completed in the queue, its status becomes 'Completed'. When the manager opens the Billing tab, Billing_DAO.autoCreateBills() executes an INSERT INTO bills ... SELECT query that automatically generates an unpaid invoice for any completed service record that doesn't yet have an invoice. The manager can then add extra parts, apply discounts, collect payment, and print the receipt."*

Q8: How did you fix the login screen centering issue on full screen?
Your Answer: *"In the NetBeans Matisse design section of LoginForm.form and initComponents(), the top margin was previously hardcoded as a fixed 75px gap while the bottom was flexible. I updated the top gap to be a flexible container gap with pref=75, max=Short.MAX_VALUE. Now both top and bottom margins expand equally like springs, keeping the login card perfectly centered on any resolution."*

Q9: Why are there 9 tables in your database instead of 11?
Your Answer: *"During our code and database audit, we verified that the payments table and service_record_items table had 0 rows and were never referenced in our Java source code. Our application records payment status directly on the bills table and stores service descriptions in service_records. To maintain a clean, normalized, and production-ready schema, we dropped those two redundant tables."*

Q10: What third-party libraries did you use?
Your Answer: *"We used two libraries: FlatLaf 3.7.2 for modern, rounded, clean UI styling, and MySQL Connector/J 26.7.0 for high-performance JDBC connectivity with MySQL."*

All Systems Ready for Evaluation

CarCare Auto Workshop Management System • Developed by Muhammad • ColonNext Internship